

Title: Automation Testing Requirements for Enterprise Dashboard Application

1. Project Overview

This document defines the requirements for implementing a UI and functional automation testing framework for a web-based enterprise application that includes dashboards, tables, charts, and workflow wizards. The application uses SSO-based login and implements Role-Based Access Control (RBAC).

2. Application Tech Stack

Layer	Technology Used
-------	-----------------

Frontend	Next.js (TypeScript, App Router)
State Mgmt	Redux Toolkit
Backend	Java Spring Boot (REST API)
Database	Snowflake
Auth	SSO + Role-Based Authentication
CI/CD	GitHub Actions + AWS

3. Automation Tools Chosen

For Option 1: Selenium with Java

Java

Selenium WebDriver

TestNG / JUnit

Page Object Model

Extent Reports / Allure Reports

Maven / Gradle

For Option 2: Selenium with Robot Framework

Robot Framework

SeleniumLibrary

Python

Keyword-Driven Test Design

Allure Robot Framework plugin

4. Core Testing Requirements

4.1 Functional Coverage

Login (via SSO)

Role-based navigation and visibility

Dashboards (charts rendering, data loading, filters)

Tables (sorting, filtering, pagination)

Workflow Wizard (multi-step form validation, submission, approval/rejection)

4.2 Non-Functional Coverage

Cross-browser compatibility

Role-based accessibility checks

Regression and Smoke Testing

4.3 Integration Requirements

GitHub Actions to trigger automated test runs

Allure / HTML reports

Test environment selection via parameters (dev, QA, staging)

5. Framework Requirements

Common

Modular structure supporting page-specific scripts/resources

Configurable environment and credentials

Parameterized tests (for roles, browsers, environments)

Reusable locators and keywords (Robot) or POM classes (Java)

Selenium with Java Framework Structure

```
java-tests/  
├── src/  
│   ├── base/  
│   ├── pages/  
│   ├── tests/  
│   ├── utils/  
│   └── resources/  
└── testng.xml / junit-config.xml
```

Robot Framework Structure

```
robot-tests/  
├── resources/  
├── testcases/  
├── variables/  
├── results/  
└── robot.yaml (GitHub Actions)
```

6. Test Case Requirements

SSO Login Tests

Successful login with valid credentials

Failed login with invalid credentials

Redirection and session timeout handling

RBAC Tests

Menu visibility by role

Access restrictions to unauthorized sections

Dashboard Tests

Chart data validation

Filtering and date range selectors

Table sorting, pagination, and export buttons

Workflow Tests

Create new workflow (all steps)

Error handling and required fields

Workflow submission and approval

7. Reporting Requirements

Dashboard-style test execution reports (Allure/Extent)

Screenshots on failure

Summary and log output (per test)

8. CI/CD Integration Requirements

GitHub Actions should trigger tests on every PR merge to main/dev

Store test artifacts (screenshots, reports) in S3 or GitHub artifacts

Provide Slack/email notifications on failure

9. Resource Requirements

Skill Set

Role Skills Needed

QA Automation Lead	Selenium, GitHub Actions, Reporting
Test Engineers	Java / Python, Selenium, Test Planning
DevOps	CI/CD config with GitHub & AWS
Developers	Add test IDs and locators in UI code

Tools

IDE: IntelliJ / VSCode

Browsers: Chrome, Firefox

GitHub Repos for tests and actions

Test Environment with SSO enabled

10. Timeline & Milestones

Phase Timeline (Weeks)

Requirement Finalization	1	
Framework Setup	1	
Smoke Test Case Automation		1
Full Regression Coverage	2–3	
CI/CD Integration	1	
Review & Handoff	1	

11. Deliverables

Framework code repository

Test suites and resource files

CI/CD pipeline integration

Test execution reports

Team onboarding guide

Let me know if you want this broken down into individual Jira tickets or a starter GitHub repo for either framework.

Here's a well-structured requirements document titled "Selenium Test Automation Requirements", incorporating all our previous discussions and tailored for both Selenium with Java and Selenium with Robot Framework approaches. It includes:

Full tech stack alignment

Framework structure

Detailed test case categories

CI/CD strategy

Timeline and team structure

Let me know if you want this split into individual tasks for your team or need a sample GitHub repo to start from.